

loop. A *factorialByIteration.py* program looks like this:

Python

```
def factorial(number):
    product = 1
    for i in range(1, number + 1):
        product = product * i
    return product
print(factorial(5))
```

And a *factorialByIteration.html* program looks like this:

JavaScript

```
<script type="text/javascript">
function factorial(number) {
    let product = 1;
    for (let i = 1; i <= number; i++) {
        product = product * i;
    }
    return product;
}
document.write(factorial(5));
</script>
```

When you run this code, the output displays the calculation for 5! like this:

120

There's nothing wrong with the iterative solution for calculating factorials; it's straightforward and gets the job done. But let's also take a look at the recursive algorithm for insights into the nature of factorials and recursion itself.

The Recursive Factorial Algorithm

Notice that the factorial of 4 is $4 \times 3 \times 2 \times 1$, and the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1$. So you could say that $5! = 5 \times 4!$. This is *recursive* because the definition of the factorial of 5 (or any number n) includes the definition of